

FIRAT ÜNİVERSİTESİ
Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü

BMÜ329 VERİ TABANI SİSTEMLERİ
DÖNEM PROJESİ RAPORU

[BİTKİ HASTALIKLARI TAKİP SİSTEMİ]

Grup No	[11.Grup]
Takım Üyesi 1	[245260020] - [Ecenur Çevik]
Takım Üyesi 2	[245260018] - [Ahmet Kurt]
Takım Üyesi 3	[235260006] - [Zeynep Sevim Özkök]
Ders Sorumlusu	[Dr. Öğr. Üyesi Ertan Bütün]
Teslim Tarihi	[27/12/2025]

İÇİNDEKİLER

GİRİŞ VE PROJE TANIMI

- 1.1 Projenin Amacı ve Kapsamı
- 1.2 Hedef Kullanıcılar

PROJE GEREKSİNİMLERİ

- 2.1 Fonksiyonel Gereksinimler
- 2.2 Fonksiyonel Olmayan Gereksinimler
- 2.3 İş Kuralları ve Kısıtlamalar

VERİ TABANI TASARIMI – VARLIK-İLİŞKİ (E-R) MODELİ

- 3.1 E-R Diyagramı

İLİŞKİSEL ŞEMALAR

- 4.1 E-R'dan İlişkisel Şemalara Dönüşüm

NORMALİZASYON

- 5.1 Normalizasyon Süreci
- 5.2 Son Normalize Edilmiş Şema

SQL SERVER VERİ TABANI ŞEMASI

- 6.1 Tablo Oluşturma Komutları
- 6.2 Birincil ve Yabancı Anahtarlar
- 6.3 Veri Tabanı Diyagramı

ÖRNEK VERİLER

- 7.1 Veri Ekleme Komutları (INSERT)

SQL KOMUTLARI VE SCRIPT DOSYALARI

- 8.1 Gereksinim Bazlı SQL Komutları

SAKLI YORDAM VE TETİKLEYİCİLER

9.1 Saklı Yordamlar (Stored Procedures)

9.2 Tetikleyiciler (Triggers)

TRANSACTION YÖNETİMİ

10.1 Tedavi ve İlaç Kayıtlarının Birlikte Oluşturulması

10.2 Transaction Kodları

10.3 Başarılı Senaryo Testleri

10.4 Hata Senaryosu ve ROLLBACK Testleri

TAKIM ÇALIŞMASI VE GÖREV DAĞILIMI

11.1 Takım Üyeleri ve Roller

11.2 Gerçekleştirilen İşler ve Sorumluluk Matrisi

SONUÇ VE DEĞERLENDİRME

1. GİRİŞ VE PROJE TANIMI

1.1 Projenin Amacı ve Kapsamı

Günümüzde tarımsal üretimde karşılaşılan en önemli sorunlardan biri, bitkilerde meydana gelen hastalıkların zamanında tespit edilememesi ve bu hastalıklara yönelik uygulanan tedavi süreçlerinin sistematik olarak takip edilememesidir. Bu durum hem verim kaybına hem de gereksiz ilaç kullanımına yol açabilmektedir. Bu proje kapsamında geliştirilen Bitki Hastalıkları Takip Sistemi, bitkilerde görülen hastalıkların, hastalığa neden olan patojenlerin, ortaya çıkan belirtilerin ve uygulanan tedavi yöntemlerinin düzenli ve bütüncül bir şekilde kayıt altına alınmasını amaçlamaktadır.

Sistem, kullanıcıların sahip oldukları bitkileri lokasyon bilgileriyle birlikte tanımlayabilmelerine, bitkiler üzerinde oluşan hastalıkları ve bu hastalıklara ait belirtileri detaylı olarak kaydedebilmelerine olanak tanımaktadır. Ayrıca her hastalık için uygulanan tedavi yöntemleri ve kullanılan ilaçlar ayrı tablolar aracılığıyla izlenebilmektedir. Bitkinin farklı bölgelerinden alınan toprak analiz sonuçları raporlanarak, bitki sağlığı ile toprak özellikleri arasındaki ilişki de sistem üzerinden takip edilebilmektedir.

Proje kapsamında geliştirilen veritabanı yapısı; kullanıcı, bitki, hastalık, patojen, belirti, tedavi, ilaç, toprak analizi, sulama ve gübreleme gibi temel bileşenleri içermektedir. Böylece bitkinin hastalık süreci başlangıcından tedavi aşamasına kadar geçen tüm adımlar kayıt altına alınmaktadır.

Bu sistem, yalnızca hastalık takibi ve raporlama süreçlerini kapsamaktadır. Gerçek zamanlı sensör verileri, otomatik hastalık teşhisi veya görüntü işleme tabanlı analizler proje kapsamı dışında bırakılmıştır. Projenin temel sınırı, kullanıcı tarafından girilen veriler üzerinden bitki hastalıklarının yönetilmesi ve geçmiş kayıtların analiz edilebilir şekilde saklanmasıdır.

1.2 Hedef Kullanıcılar

Bitki Hastalıkları Takip Sistemi, tarımsal üretim süreçlerinde aktif rol alan farklı kullanıcı gruplarına hitap edecek şekilde tasarlanmıştır. Sistem içerisinde yer alan başlıca hedef kullanıcılar aşağıda belirtilmiştir:

- **Çiftçiler ve Üreticiler:** Sahip oldukları bitkileri sisteme kaydederek, bitkilerde görülen hastalıkları, uygulanan tedavileri ve kullanılan ilaçları düzenli olarak takip edebilirler.
- **Ziraat Mühendisleri ve Uzmanlar:** Bitkilerde oluşan hastalıkları, patojen türlerini ve belirtileri inceleyerek doğru tedavi yöntemlerinin belirlenmesine katkı sağlayabilirler.
- **Tarım Danışmanları:** Farklı bitkiler ve lokasyonlar için geçmiş hastalık ve tedavi kayıtlarını analiz ederek üreticilere önerilerde bulunabilirler.

- **Sistem Yöneticileri:** Kullanıcı yönetimi, veri bütünlüğü ve raporların kontrolünden sorumlu olan yetkili kullanıcılardır.

Bu kullanıcı grupları sayesinde sistem, hem bireysel üreticiler hem de tarımsal danışmanlık hizmeti veren profesyoneller için etkin bir takip ve raporlama aracı olarak kullanılabilir.

2. PROJE GEREKSİNİMLERİ

Bu bölümde Bitki Hastalıkları Takip Sistemi için belirlenen fonksiyonel ve fonksiyonel olmayan gereksinimler ile birlikte, sistemin işleyişini düzenleyen iş kuralları ve kısıtlamalar açıklanmaktadır.

2.1 Fonksiyonel Gereksinimler

ID	Gereksinim Adı	Açıklama
FR-01	Kullanıcı Yönetimi	Sistem, farklı rollere sahip kullanıcıların (çiftçi, uzman, yönetici vb.) kayıt edilmesini ve yetkilerine göre işlem yapabilmesini sağlamalıdır.
FR-02	Bitki Tanımlama	Kullanıcılar, sahip oldukları bitkileri lokasyon bilgileriyle birlikte sisteme ekleyebilmelidir.
FR-03	Hastalık Kaydı	Bir bitki üzerinde görülen hastalıklar, hastalığın bilimsel adı ve patojen bilgisi ile birlikte sisteme kaydedilebilmelidir.
FR-04	Belirti Takibi	Her hastalık için gözlemlenen bir veya birden fazla belirti sisteme tanımlanabilmelidir.
FR-05	Tedavi ve İlaç Yönetimi	Hastalıklar için uygulanan tedavi yöntemleri ve kullanılan ilaçlar doz bilgileriyle birlikte kayıt altına alınabilmelidir.
FR-06	Toprak Analizi Kaydı	Bitkinin farklı bölgelerinden alınan toprak analiz sonuçları sisteme girilebilmelidir.
FR-07	Sulama ve Gübreleme Takibi	Bitkiler için yapılan sulama ve gübreleme işlemleri tür, doz ve süre bilgileriyle izlenebilmelidir.
FR-08	Vaka ve Rapor Oluşturma	Sistem, hastalık sürecine ait vaka kayıtlarını ve bu vakalara bağlı raporları oluşturabilmelidir.

2.2 Fonksiyonel Olmayan Gereksinimler

Sistemin güvenilir, sürdürülebilir ve doğru çalışmasını sağlamak amacıyla belirlenen fonksiyonel olmayan gereksinimler aşağıda sunulmuştur.

ID	Gereksinim Adı	Açıklama
NFR-01	Güvenlik	Kullanıcı bilgileri ve sistem verileri yetkisiz erişimlere karşı korunmalı, erişim yetkileri rol bazlı olarak sınırlandırılmalıdır.
NFR-02	Veri Bütünlüğü	Veritabanında referans bütünlüğü sağlanmalı, yabancı anahtar (foreign key) kısıtlamaları ile tutarsız veri girişleri engellenmelidir.
NFR-03	Performans	Sistem, çoklu kullanıcı erişimlerinde temel sorguları kabul edilebilir sürede sonuçlandırabilmelidir.
NFR-04	Yedekleme	Veritabanı, olası veri kayıplarına karşı düzenli olarak yedeklenebilir yapıda olmalıdır.
NFR-05	Ölçeklenebilirlik	Sistem tasarımı, ileride yeni özelliklerin veya tabloların eklenmesine olanak tanıyacak şekilde planlanmalıdır.

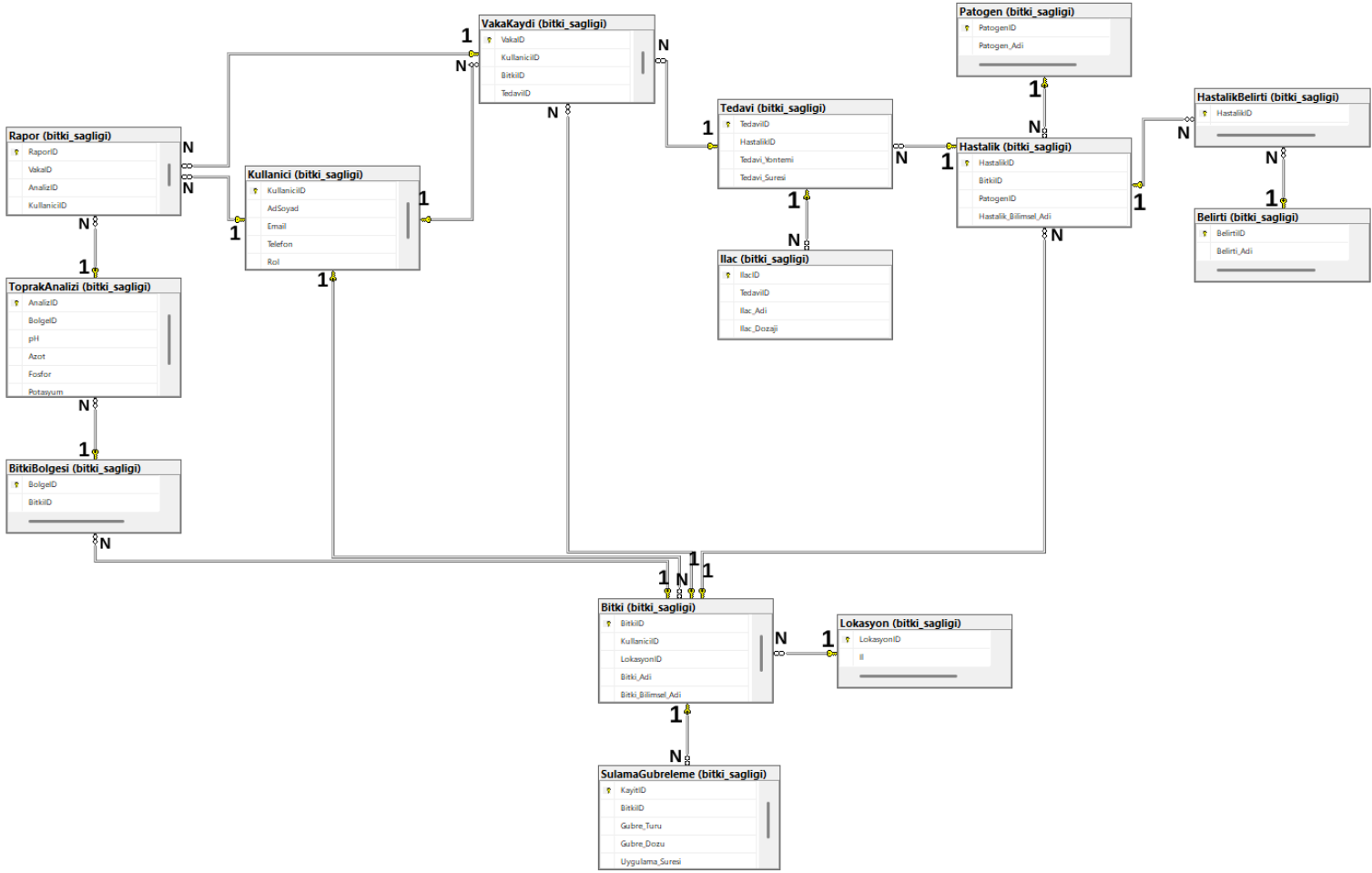
2.2 İş Kuralları, Kısıtlamalar

Sistemin doğru ve tutarlı çalışmasını sağlamak amacıyla aşağıdaki iş kuralları ve kısıtlamalar tanımlanmıştır:

- Aynı bitki için aynı anda birden fazla aktif hastalık kaydı oluşturulabilir; ancak her hastalık kaydı belirli bir patojen ile ilişkilendirilmelidir.
- Bir hastalık, en az bir belirti ile ilişkilendirilmeden sisteme kaydedilemez.
- Her tedavi kaydı mutlaka bir hastalığa bağlı olmalıdır.
- Bir ilaç kaydı, yalnızca tanımlı bir tedavi yöntemi üzerinden sisteme eklenebilir.
- Toprak analizleri yalnızca tanımlı bitki bölgeleri için yapılabilir.
- Sulama ve gübreleme kayıtları, ilgili bitki tanımlanmadan sisteme eklenemez.
- Silinen bir bitkiyle ilişkili hastalık, tedavi ve analiz kayıtları veri bütünlüğünü koruyacak şekilde kısıtlanmalıdır.

3. VERİ TABANI TASARIMI - VARLIK-İLİŞKİ (E-R) MODELİ

3.1 E-R Diyagramı



4. İLİŞKİSEL ŞEMALAR

4.1 E-R'dan İlişkisel Şemalarına Dönüşüm

KULLANICI(**KullaniciID**, AdSoyad, Email, Telefon, Rol)

LOKASYON(**LokasyonID**, İl, İlçe)

BITKİ(**BitkiID**, KullaniciID, LokasyonID, Bitki_Adi, Bitki_Bilimsel_Adi)

PATOGEN(**PatogenID**, Patogen_Adi, Patogen_Turu)

BELİRTİ(**BelirtiID**, Belirti_Adi, Belirti_Tarihi)

HASTALIK(**HastalikID**, BitkiID, PatogenID, Hastalik_Bilimsel_Adi)

HASTALIK_BELİRTİ(HastalikID, BelirtiID)

TEDAVİ(**TedaviID**, HastalikID, Tedavi_Yontemi, Tedavi_Suresi)

ILAC(**IlacID**, TedaviID, Ilac_Adi, Ilac_Dozaji)

BITKİ_BOLGESİ(**BolgeID**, BitkiID, Bitki_Bolge_Adi)

TOPRAK_ANALİZİ(**AnalizID**, BolgeID, pH, Azot, Fosfor, Potasyum)

VAKA_KAYDI(**VakaID**, KullaniciID, BitkiID, HastalikID, LokasyonID, TedaviID)

RAPOR(**RaporID**, VakaID, AnalizID, KullaniciID)

SULAMA_GUBRELEME(**KayitID**, BitkiID, Gubre_Turu, Gubre_Dozu, Uygulama_Suresi)

5. NORMALİZASYON

5.1 Normalizasyon Süreci

[Mümkünse BCNF değilse 3NF'e normalizasyon sürecini adım adım gösteriniz.]

NOT

R12 VakaKaydi ilişkisi anahtar olmayan belirleyiciler içerdiği için BCNF koşulunu sağlamamakta ve bu nedenle normalizasyon sürecinde ayrı olmak üzere ele alınmıştır.

R1 Kullanici

KullaniciID → AdSoyad, Email, Telefon, Rol

Email → KullaniciID, AdSoyad, Telefon, Rol

KullaniciID ve Email aday anahtar → BCNF

R1 Kullanici ilişkisinde KullaniciID birincil anahtar olup tüm öznitelikleri belirlemektedir. Ayrıca Email alanı UNIQUE kısıtı ile tanımlandığından aday anahtar niteliği taşımaktadır. İlişkide tanımlı olan tüm işlevsel bağımlılıkların belirleyici tarafı anahtar veya aday anahtar olduğundan, anahtar olmayan bir belirleyici bulunmamaktadır. Bu nedenle R1 ilişkisi BCNF'dedir.

R2 Lokasyon

LokasyonID → İl, İlçe

(İl, İlçe) → LokasyonID

LokasyonID ve (İl, İlçe) aday anahtar → BCNF

R2 Lokasyon ilişkisinde LokasyonID birincil anahtar olarak tüm öznitelikleri belirlemektedir. Ayrıca İl ve İlçe özniteliklerinden oluşan birleşik alan UNIQUE kısıtı ile tanımlandığı için aday anahtar niteliğindedir. Tüm işlevsel bağımlılıkların belirleyici tarafı anahtar olduğundan, R2 ilişkisi BCNF koşulunu sağlamaktadır.

R3 Bitki

BitkiID → KullaniciID, LokasyonID, Bitki_Adi, Bitki_Bilimsel_Adi

R3 Bitki ilişkisinde BitkiID birincil anahtar olup ilişkideki tüm öznitelikleri belirlemektedir. KullaniciID ve LokasyonID öznitelikleri yabancı anahtar olmasına rağmen belirleyici değildir. Anahtar olmayan herhangi bir öznitelik başka öznitelikleri belirlemediği için R3 ilişkisi BCNF'dedir.

Belirleyen sadece BitkiID (anahtar) → BCNF

R4 Patogen

PatogenID → Patogen_Adi, Patogen_Turu

(Patogen_Adi, Patogen_Turu) → PatogenID

PatogenID ve (Patogen_Adi, Patogen_Turu) aday anahtar → BCNF

R4 Patogen ilişkisinde PatogenID birincil anahtar olarak tüm öznitelikleri belirlemektedir. Ayrıca Patogen_Adi ve Patogen_Turu özniteliklerinin birleşimi aday anahtar niteliğindedir. Tüm belirleyiciler anahtar veya aday anahtar olduğu için ilişki BCNF'dedir.

R5 Belirti

BelirtiID → Belirti_Adi, Belirti_Tarihi

BelirtiID aday anahtar → BCNF

R5 Belirti ilişkisinde BelirtiID birincil anahtar olup ilişkideki tüm öznitelikleri belirlemektedir. Tanımlanan işlevsel bağımlılıkta belirleyici olan BelirtiID anahtar niteliğinde olduğundan, anahtar olmayan herhangi bir öznitelik başka öznitelikleri belirlememektedir. Bu nedenle R5 Belirti ilişkisi BCNF koşulunu sağlamaktadır.

R6 Hastalik

HastalikID → BitkiID, PatogenID, Hastalik_Bilimsel_Adi

Belirleyen sadece HastalikID → BCNF

R6 Hastalik ilişkisinde HastalikID birincil anahtar olup tüm öznitelikleri belirlemektedir. Anahtar olmayan herhangi bir belirleyici bulunmadığından ilişki BCNF koşulunu sağlamaktadır.

R7 HastalikBelirti

(HastalikID, BelirtiID) → ∅

Bileşik anahtar var, başka FD yok → BCNF

R7 HastalikBelirti ilişkisinde HastalikID ve BelirtiID özniteliklerinden oluşan birleşik anahtar bulunmaktadır. Anahtar dışında başka bir belirleyici olmadığı ve ek öznitelik içermediği için ilişki BCNF'dedir.

R8 Tedavi

TedaviID → HastalikID, Tedavi_Yontemi, Tedavi_Suresi

Belirleyen sadece TedaviID → BCNF

R8 Tedavi ilişkisinde TedaviID birincil anahtar olarak tüm öznitelikleri belirlemektedir. Anahtar olmayan bir belirleyici bulunmadığı için ilişki BCNF koşulunu sağlamaktadır.

R9 Ilac

IlacID → TedaviID, Ilac_Adi, Ilac_Dozaji

Belirleyen sadece IlacID → BCNF

R9 Ilac ilişkisinde IlacID birincil anahtar olup tüm öznitelikleri belirlemektedir. İlişkide anahtar olmayan bir belirleyici yer almadığından BCNF sağlanmaktadır.

R10 BitkiBolgesi

BolgeID → BitkiID, Bitki_Bolge_Adi

Belirleyen sadece BolgeID → BCNF

R10 BitkiBolgesi ilişkisinde BolgeID birincil anahtar olarak tüm öznitelikleri belirlemektedir. Başka bir belirleyici bulunmadığı için ilişki BCNF'dedir.

R11 ToprakAnalizi

AnalizID → BolgeID, pH, Azot, Fosfor, Potasyum

Belirleyen sadece AnalizID → BCNF

R11 ToprakAnalizi ilişkisinde AnalizID birincil anahtar olup tüm öznitelikleri belirlemektedir. Anahtar olmayan herhangi bir belirleyici bulunmadığından ilişki BCNF koşulunu sağlamaktadır.

R13 Rapor

RaporID → VakaID, AnalizID, KullaniciID

Belirleyen sadece RaporID → BCNF

R13 Rapor ilişkisinde RaporID birincil anahtar olarak tüm öznitelikleri belirlemektedir. İlişkide anahtar olmayan bir belirleyici bulunmadığı için BCNF sağlanmaktadır.

R14 SulamaGubreleme

KayitID → BitkiID, Gubre_Turu, Gubre_Dozu, Uygulama_Suresi

Belirleyen sadece KayitID → BCNF

R14 SulamaGubreleme ilişkisinde KayitID birincil anahtar olup tüm öznitelikleri belirlemektedir. Anahtar olmayan herhangi bir belirleyici bulunmadığından ilişki BCNF'dedir.

Peki, R12 NEDEN BCNF DEĞİLDİR?

R12 VakaKaydi

İşlevsel Bağımlılıklar

VakaID → KullaniciID, BitkiID, HastalikID, LokasyonID, TedaviID

TedaviID → HastalikID

BitkiID → LokasyonID

VakaID aday anahtardır. Bu İşlevsel Bağımlılık için Sorun Yoktur.

Fakat Aşağıdaki İşlevsel Bağımlılıklarda Sorun Vardır.

TedaviID → HastalikID bağımlılığında belirleyici olan **TedaviID**, R12 ilişkisinin anahtarı değildir.

BitkiID → LokasyonID bağımlılığında belirleyici olan **BitkiID**, R12 ilişkisinin anahtarı değildir.

BCNF haline getirme

R12'deki tekrar ve geçişliliğe sebep olan öznelilikler(sütunlar), zaten başka ilişkiler üzerinden elde edildiği için kaldırabiliriz:

TedaviID → HastalikID olduğundan **HastalikID**, Tedavi tablosu üzerinden bulunabilir → R12'den çıkarılır.

BitkiID → LokasyonID olduğundan **LokasyonID**, Bitki tablosu üzerinden bulunabilir → R12'den çıkarılır.

Bu ayrıştırma sonucunda elde edilen yeni ilişki:

R12' VakaKaydi'(VakaID, KullaniciID, BitkiID, TedaviID)

Bu yeni ilişkide belirleyici olan tek öznelilik **VakaID** (anahtar) olduğundan **BCNF koşulu sağlanır**.

5.2 Son Normalize Edilmiş Şema

R1 Kullanici(KullaniciID, AdSoyad, Email, Telefon, Rol)

R2 Lokasyon(LokasyonID, İl, İlçe)

R3 Bitki(BitkiID, KullaniciID, LokasyonID, Bitki_Adi, Bitki_Bilimsel_Adi)

R4 Patogen(PatogenID, Patogen_Adi, Patogen_Turu)

R5 Belirti(BelirtiID, Belirti_Adi, Belirti_Tarihi)

R6 Hastalık(HastalikID, BitkiID, PatogenID, Hastalik_Bilimsel_Adi)

R7 HastalikBelirti(HastalikID, BelirtiID)

R8 Tedavi(TedaviID, HastalikID, Tedavi_Yontemi, Tedavi_Suresi)

R9 Ilac(IlacID, TedaviID, Ilac_Adi, Ilac_Dozaji)

R10 BitkiBolgesi(BolgeID, BitkiID, Bitki_Bolge_Adi)

R11 ToprakAnalizi(AnalizID, BolgeID, pH, Azot, Fosfor, Potasyum)

R12' VakaKaydi'(VakaID, KullaniciID, BitkiID, TedaviID)

R13 Rapor(RaporID, VakaID, AnalizID, KullaniciID)

R14 SulamaGubreleme(KayitID, BitkiID, Gubre_Turu, Gubre_Dozu, Uygulama_Suresi)

6. SQL SERVER VERİ TABANI ŞEMASI

6.1 Tablo Oluşturma Komutları

```

CREATE TABLE Kullanici (
    KullaniciID INT PRIMARY KEY IDENTITY(1,1),
    AdSoyad VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Telefon VARCHAR(20),
    Rol VARCHAR(50) NOT NULL
);

CREATE TABLE Lokasyon (
    LokasyonID INT PRIMARY KEY IDENTITY(1,1),
    Il VARCHAR(50) NOT NULL,
    Ilce VARCHAR(50) NOT NULL,
    CONSTRAINT UQ_Lokasyon UNIQUE (Il, Ilce)
);

CREATE TABLE Bitki (
    BitkiID INT PRIMARY KEY IDENTITY(1,1),
    KullaniciID INT NOT NULL,
    LokasyonID INT NOT NULL,
    Bitki_Adi VARCHAR(100) NOT NULL,
    Bitki_Bilimsel_Adi VARCHAR(150) NOT NULL,
    FOREIGN KEY (KullaniciID) REFERENCES Kullanici(KullaniciID),
    FOREIGN KEY (LokasyonID) REFERENCES Lokasyon(LokasyonID)
);

CREATE TABLE Patogen (
    PatogenID INT PRIMARY KEY IDENTITY(1,1),
    Patogen_Adi VARCHAR(100) NOT NULL,
    Patogen_Turu VARCHAR(50) NOT NULL,
    CONSTRAINT UQ_Patogen UNIQUE (Patogen_Adi, Patogen_Turu)
);

CREATE TABLE Belirti (
    BelirtiID INT PRIMARY KEY IDENTITY(1,1),
    Belirti_Adi VARCHAR(100) NOT NULL,
    Belirti_Tarihi DATE NOT NULL
);

CREATE TABLE Hastalik (
    HastalikID INT PRIMARY KEY IDENTITY(1,1),
    BitkiID INT NOT NULL,
    PatogenID INT NOT NULL,
    Hastalik_Bilimsel_Adi VARCHAR(150) NOT NULL,
    FOREIGN KEY (BitkiID) REFERENCES Bitki(BitkiID),
    FOREIGN KEY (PatogenID) REFERENCES Patogen(PatogenID)
);

CREATE TABLE HastalikBelirti (
    HastalikID INT NOT NULL,
    BelirtiID INT NOT NULL,
    PRIMARY KEY (HastalikID, BelirtiID),
    FOREIGN KEY (HastalikID) REFERENCES Hastalik(HastalikID),
    FOREIGN KEY (BelirtiID) REFERENCES Belirti(BelirtiID)
);

CREATE TABLE Tedavi (
    TedaviID INT PRIMARY KEY IDENTITY(1,1),
    HastalikID INT NOT NULL,
    Tedavi_Yontemi VARCHAR(100) NOT NULL,

```

```

        Tedavi_Suresi INT NOT NULL,
        FOREIGN KEY (HastalikID) REFERENCES Hastalik(HastalikID)
    );

CREATE TABLE Ilac (
    IlacID INT PRIMARY KEY IDENTITY(1,1),
    TedaviID INT NOT NULL,
    Ilac_Adi VARCHAR(100) NOT NULL,
    Ilac_Dozaji VARCHAR(50) NOT NULL,
    FOREIGN KEY (TedaviID) REFERENCES Tedavi(TedaviID)
);

CREATE TABLE BitkiBolgesi (
    BolgeID INT PRIMARY KEY IDENTITY(1,1),
    BitkiID INT NOT NULL,
    Bitki_Bolge_Adi VARCHAR(100) NOT NULL,
    FOREIGN KEY (BitkiID) REFERENCES Bitki(BitkiID)
);

CREATE TABLE ToprakAnalizi (
    AnalizID INT PRIMARY KEY IDENTITY(1,1),
    BolgeID INT NOT NULL,
    pH FLOAT NOT NULL,
    Azot FLOAT,
    Fosfor FLOAT,
    Potasyum FLOAT,
    FOREIGN KEY (BolgeID) REFERENCES BitkiBolgesi(BolgeID)
);

CREATE TABLE VakaKaydi (
    VakaID INT PRIMARY KEY IDENTITY(1,1),
    KullaniciID INT NOT NULL,
    BitkiID INT NOT NULL,
    HastalikID INT NOT NULL,
    LokasyonID INT NOT NULL,
    TedaviID INT NOT NULL,
    FOREIGN KEY (KullaniciID) REFERENCES Kullanici(KullaniciID),
    FOREIGN KEY (BitkiID) REFERENCES Bitki(BitkiID),
    FOREIGN KEY (HastalikID) REFERENCES Hastalik(HastalikID),
    FOREIGN KEY (LokasyonID) REFERENCES Lokasyon(LokasyonID),
    FOREIGN KEY (TedaviID) REFERENCES Tedavi(TedaviID)
);

CREATE TABLE Rapor (
    RaporID INT PRIMARY KEY IDENTITY(1,1),
    VakaID INT NOT NULL,
    AnalizID INT NOT NULL,
    KullaniciID INT NOT NULL,
    FOREIGN KEY (VakaID) REFERENCES VakaKaydi(VakaID),
    FOREIGN KEY (AnalizID) REFERENCES ToprakAnalizi(AnalizID),
    FOREIGN KEY (KullaniciID) REFERENCES Kullanici(KullaniciID)
);

CREATE TABLE SulamaGubreleme (
    KayitID INT PRIMARY KEY IDENTITY(1,1),
    BitkiID INT NOT NULL,
    Gubre_Turu VARCHAR(100) NOT NULL,
    Gubre_Dozu FLOAT NOT NULL,
    Uygulama_Suresi INT NOT NULL,
    FOREIGN KEY (BitkiID) REFERENCES Bitki(BitkiID)
);

```

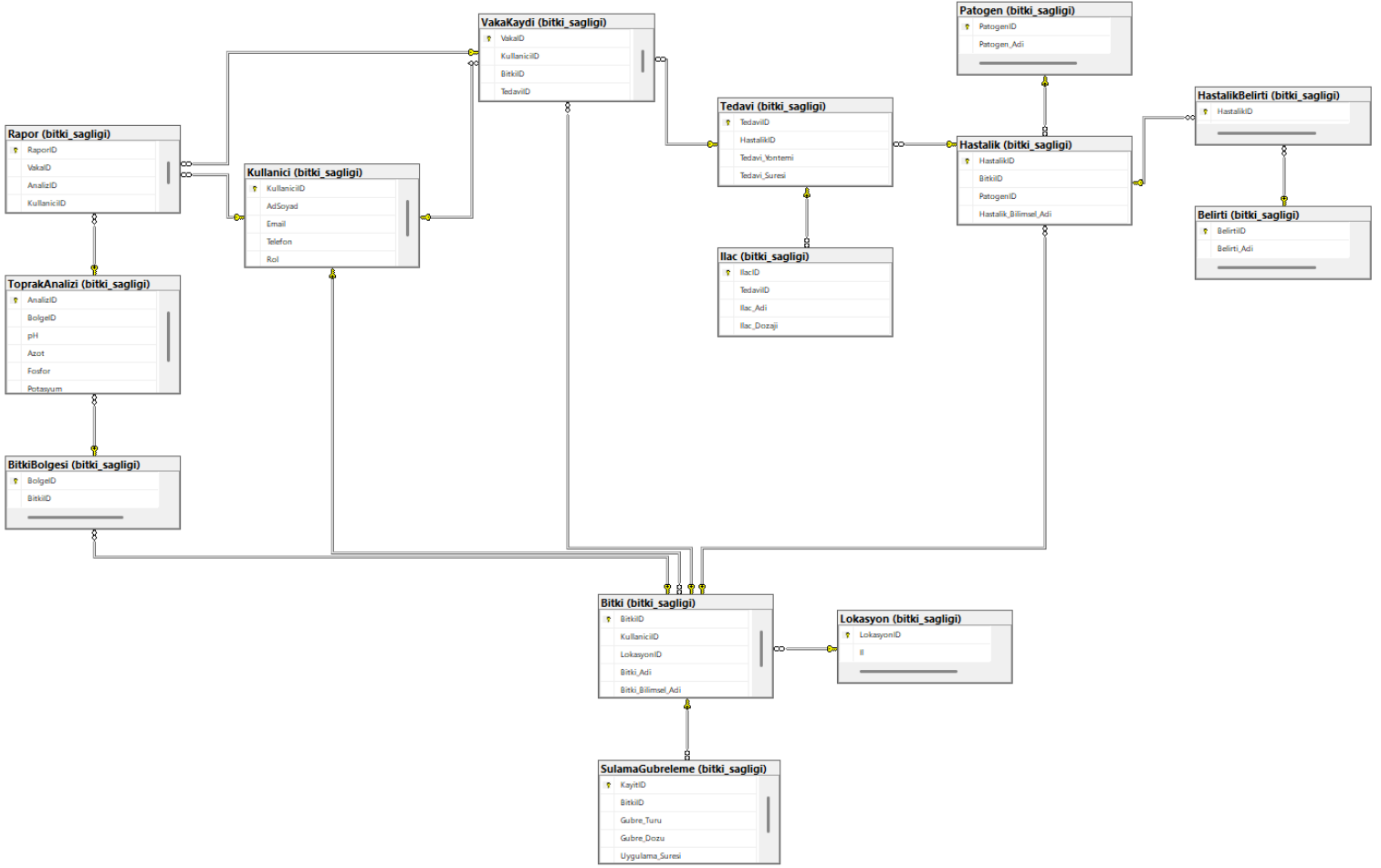

Birincil Anahtarlar (Primary Key)

Bu veritabanında her tabloda kayıtların benzersiz olarak ayırt edilebilmesi için KullanıcıID, LokasyonID, BitkiID, PatogenID, BelirtiID, HastalıkID, TedaviID, IlacID, BolgeID, AnalizID, VakaID, RaporID ve KayıtID alanlarını birincil anahtar olarak tanımladım, hastalık ile belirtiler arasındaki çoktan çoğa ilişkii göstermek için Hastalık Belirti tablosunda (HastalıkID, BelirtiID) bileşik birincil anahtar kullandım.

Yabancı Anahtarlar (Foreign Key)

Tablolar arasında ilişki kurulması ve veri bütünlüğünün sağlanması amacıyla Bitki tablosunda KullanıcıID ve LokasyonID, Hastalık tablosunda BitkiID ve PatogenID, HastalıkBelirti tablosunda HastalıkID ve BelirtiID, Tedavi tablosunda HastalıkID, Ilac tablosunda TedaviID, BitkiBolgesi tablosunda BitkiID, ToprakAnalizi tablosunda BolgeID, VakaKaydi tablosunda KullanıcıID, BitkiID, HastalıkID, LokasyonID ve TedaviID, Rapor tablosunda VakaID, AnalizID ve KullanıcıID, SulamaGubreleme tablosunda ise BitkiID alanlarını yabancı anahtar olarak tanımladım ve kullandım.

6.3 Veri Tabanı Diyagramı



7. ÖRNEK VERİLER

7.1 Veri Ekleme Komutları (INSERT)

```
-- Kullanici tablosu:
INSERT INTO bitki_sagligi.Kullanici (AdSoyad, Email, Telefon, Rol) VALUES
('Ahmet Yılmaz', 'ahmet1@mail.com', '0500000001', 'Çiftçi'),
('Mehmet Kaya', 'mehmet2@mail.com', '0500000002', 'Ziraat Mühendisi'),
('Ayşe Demir', 'ayse3@mail.com', '0500000003', 'Tarım Danışmanı'),
('Fatma Şahin', 'fatma4@mail.com', '0500000004', 'Üretici'),
('Ali Can', 'ali5@mail.com', '0500000005', 'Sistem Yöneticisi'),
('Hasan Ak', 'hasan6@mail.com', '0500000006', 'Çiftçi'),
('Zeynep Arslan', 'zeynep7@mail.com', '0500000007', 'Üretici'),
('Murat Koç', 'murat8@mail.com', '0500000008', 'Ziraat Mühendisi'),
('Elif Yıldız', 'elif9@mail.com', '0500000009', 'Tarım Danışmanı'),
('Kemal Öz', 'kemal10@mail.com', '0500000010', 'Çiftçi'),
('Burak Aydın', 'burak11@mail.com', '0500000011', 'Üretici'),
('Cem Doğan', 'cem12@mail.com', '0500000012', 'Çiftçi'),
('Ece Korkmaz', 'ece13@mail.com', '0500000013', 'Tarım Danışmanı'),
('Serkan Uslu', 'serkan14@mail.com', '0500000014', 'Ziraat Mühendisi'),
('Derya Polat', 'derya15@mail.com', '0500000015', 'Üretici'),
('Tolga Kurt', 'tolga16@mail.com', '0500000016', 'Çiftçi'),
('Nazan Acar', 'nazan17@mail.com', '0500000017', 'Sistem Yöneticisi'),
('Onur Şimşek', 'onur18@mail.com', '0500000018', 'Sistem Yöneticisi'),
('Hakan Yalçın', 'hakan19@mail.com', '0500000019', 'Çiftçi'),
('Selin Er', 'selin20@mail.com', '0500000020', 'Tarım Danışmanı'),
('Emre Kar', 'emre21@mail.com', '0500000021', 'Üretici'),
('Berk Güneş', 'berk22@mail.com', '0500000022', 'Çiftçi'),
('Seda Mutlu', 'seda23@mail.com', '0500000023', 'Ziraat Mühendisi'),
('Volkan Işık', 'volkan24@mail.com', '0500000024', 'Çiftçi'),
('Merve Toprak', 'merve25@mail.com', '0500000025', 'Üretici');

--Lokasyon Tablosu

INSERT INTO bitki_sagligi.Lokasyon (Il, Ilce) VALUES
('Adana', 'Seyhan'), ('Adana', 'Yüreğir'), ('Mersin', 'Tarsus'),
('Antalya', 'Serik'), ('Konya', 'Meram'), ('Konya', 'Selçuklu'),
('Manisa', 'Akhisar'), ('İzmir', 'Tire'), ('İzmir', 'Ödemiş'),
('Aydın', 'Nazilli'), ('Bursa', 'Karacabey'), ('Balıkesir', 'Edremit'),
('Çanakkale', 'Biga'), ('Tekirdağ', 'Malkara'), ('Samsun', 'Bafra'),
('Tokat', 'Erbaa'), ('Kayseri', 'Develi'), ('Sivas', 'Şarkışla'),
('Malatya', 'Battalgazi'), ('Elazığ', 'Merkez'),
('Şanlıurfa', 'Harran'), ('Gaziantep', 'Nizip'),
('Diyarbakır', 'Bismil'), ('Mardin', 'Kızıltepe'),
('Hatay', 'Kırıkhan');

--Bitki Tablosu

INSERT INTO bitki_sagligi.Bitki (KullaniciID, LokasyonID, Bitki_Adi,
Bitki_Bilimsel_Adi) VALUES
(1,1, 'Buğday', 'Triticum aestivum'),
(2,2, 'Mısır', 'Zea mays'),
(3,3, 'Domates', 'Solanum lycopersicum'),
(4,4, 'Biber', 'Capsicum annuum'),
(5,5, 'Patates', 'Solanum tuberosum'),
(6,6, 'Pamuk', 'Gossypium hirsutum'),
(7,7, 'Zeytin', 'Olea europaea'),
(8,8, 'Üzüm', 'Vitis vinifera'),
(9,9, 'Elma', 'Malus domestica'),
(10,10, 'Arpa', 'Hordeum vulgare'),
(11,11, 'Ayçiçeği', 'Helianthus annuus'),
```

```
(12,12,'Soğan','Allium cepa'),
(13,13,'Sarımsak','Allium sativum'),
(14,14,'Pirinç','Oryza sativa'),
(15,15,'Kavun','Cucumis melo'),
(16,16,'Karpuz','Citrullus lanatus'),
(17,17,'Nohut','Cicer arietinum'),
(18,18,'Mercimek','Lens culinaris'),
(19,19,'Fasulye','Phaseolus vulgaris'),
(20,20,'Çilek','Fragaria × ananassa'),
(21,21,'Pamuk','Gossypium'),
(22,22,'Limon','Citrus limon'),
(23,23,'Portakal','Citrus sinensis'),
(24,24,'Mandalina','Citrus reticulata'),
(25,25,'Nar','Punica granatum');
```

--Bitkilerin Hasarlı Bölgesinin Olduğu Tablo

```
INSERT INTO bitki_sagligi.BitkiBolgesi (BitkiID, Bitki_Bolge_Adi) VALUES
(1,'Yaprak'), (2,'Gövde'), (3,'Kök'), (4,'Yaprak'),
(5,'Gövde'), (6,'Kök'), (7,'Yaprak'), (8,'Gövde'),
(9,'Kök'), (10,'Yaprak'), (11,'Gövde'), (12,'Kök'),
(13,'Yaprak'), (14,'Gövde'), (15,'Kök'),
(16,'Yaprak'), (17,'Gövde'), (18,'Kök'),
(19,'Yaprak'), (20,'Gövde'), (21,'Kök'),
(22,'Yaprak'), (23,'Gövde'), (24,'Kök'),
(25,'Yaprak');
```

--Bölgenin Toprak Analizi Tablosu

```
INSERT INTO bitki_sagligi.ToprakAnalizi (BolgeID, pH, Azot, Fosfor, Potasyum)
VALUES
(1,6.5,2.1,1.3,2.0), (2,6.8,2.3,1.5,2.2),
(3,7.0,1.9,1.2,1.8), (4,6.6,2.0,1.4,2.1),
(5,6.7,2.2,1.6,2.3), (6,6.4,1.8,1.1,1.7),
(7,6.9,2.4,1.7,2.4), (8,7.1,2.0,1.3,2.0),
(9,6.3,1.7,1.0,1.6), (10,6.8,2.1,1.4,2.2),
(11,7.0,2.5,1.8,2.6), (12,6.6,2.0,1.3,2.1),
(13,6.9,2.3,1.6,2.4), (14,7.2,2.6,1.9,2.7),
(15,6.5,1.9,1.2,1.9),
(16,6.7,2.2,1.5,2.3), (17,6.8,2.1,1.4,2.2),
(18,6.4,1.8,1.1,1.7), (19,7.0,2.4,1.7,2.5),
(20,6.6,2.0,1.3,2.1),
(21,6.9,2.3,1.6,2.4), (22,7.1,2.5,1.8,2.6),
(23,6.7,2.1,1.4,2.2), (24,6.8,2.2,1.5,2.3),
(25,7.0,2.4,1.7,2.5);
```

--Patogen Tablosu

```
INSERT INTO bitki_sagligi.Patogen (Patogen_Adi, Patogen_Turu) VALUES
('Fusarium','Mantar'), ('Phytophthora','Mantar'),
('Pseudomonas','Bakteri'), ('Xanthomonas','Bakteri'),
('CMV','Virüs'), ('TMV','Virüs'),
('Alternaria','Mantar'), ('Rhizoctonia','Mantar'),
('Erwinia','Bakteri'), ('PVY','Virüs'),
('Botrytis','Mantar'), ('Verticillium','Mantar'),
('Agrobacterium','Bakteri'), ('TSWV','Virüs'),
('Oidium','Mantar'), ('Puccinia','Mantar'),
('Clavibacter','Bakteri'), ('SMV','Virüs'),
('Colletotrichum','Mantar'), ('Ralstonia','Bakteri'),
('BYDV','Virüs'), ('Sclerotinia','Mantar'),
('Dickeya','Bakteri'), ('TYLCV','Virüs'),
('Ustilago','Mantar');
```

--Hastalik Tablosu

```

INSERT INTO bitki_sagligi.Hastalik (BitkiID, PatogenID, Hastalik_Bilimsel_Adi)
VALUES
(1,1,'Fusarium Solgunluğu'),
(2,2,'Kök Çürüklüğü'),
(3,7,'Yaprak Lekesi'),
(4,11,'Gri Küf'),
(5,8,'Kök Çürüklüğü'),
(6,16,'Pas Hastalığı'),
(7,15,'Külleme'),
(8,19,'Antraknoz'),
(9,12,'Solgunluk'),
(10,25,'Sürme'),
(11,22,'Beyaz Çürüklük'),
(12,9,'Yumuşak Çürüklük'),
(13,3,'Bakteriyel Yanıklık'),
(14,14,'Bronzlaşma'),
(15,5,'Mozaik'),
(16,10,'Virüs Hastalığı'),
(17,18,'Mozaik'),
(18,20,'Bakteriyel Solgunluk'),
(19,17,'Yanıklık'),
(20,24,'Sararma');

```

--Tedavi Tablosu

```

INSERT INTO bitki_sagligi.Tedavi (HastalikID, Tedavi_Yontemi, Tedavi_Suresi)
VALUES
(1,'Fungisit',14),(2,'Toprak Drenajı',10),
(3,'İlaçlama',7),(4,'Fungisit',10),
(5,'Toprak Dezenfeksiyonu',14),(6,'İlaçlama',7),
(7,'Kükürt',10),(8,'Fungisit',12),
(9,'Hijyen',8),(10,'İlaçlama',10),
(11,'Fungisit',14),(12,'Bakterisit',7),
(13,'Bakırlı ilaç',10),(14,'İlaçlama',7),
(15,'Zararlı Kontrolü',10),(16,'İlaçlama',7),
(17,'Virüs Kontrolü',14),(18,'Toprak İyileştirme',12),
(19,'İlaçlama',8),(20,'Besin Takviyesi',10),
(1,'Fungisit+',14),(2,'Drenaj+',10),
(3,'İlaç+',7),(4,'Fungisit+',10),
(5,'Dezenfeksiyon+',14);

```

--İlaç Tablosu

```

INSERT INTO bitki_sagligi.Ilac (TedaviID, Ilac_Adi, Ilac_Dozaji) VALUES
(1,'Fusil','2 ml/L'),(2,'RootFix','3 ml/L'),
(3,'LeafStop','1.5 ml/L'),(4,'BotryClean','2 ml/L'),
(5,'SoilCare','4 ml/L'),(6,'RustOff','2 ml/L'),
(7,'SulfurMax','3 g/L'),(8,'AnthraStop','2 ml/L'),
(9,'CleanSoil','—'),(10,'SeedProtect','1 ml/L'),
(11,'WhiteGuard','2 ml/L'),(12,'BactoKill','2 ml/L'),
(13,'CopperPlus','2.5 ml/L'),(14,'VirusStop','1 ml/L'),
(15,'MosaicFix','2 ml/L'),(16,'ViroClean','1 ml/L'),
(17,'MozaFix','2 ml/L'),(18,'WiltStop','3 ml/L'),
(19,'BurnAway','2 ml/L'),(20,'NutriBoost','5 ml/L'),
(21,'ExtraFungi','2 ml/L'),(22,'DrainMax','—'),
(23,'LeafExtra','1 ml/L'),(24,'FungiPlus','2 ml/L'),
(25,'SoilMax','4 ml/L');

```

--VakaKaydi Tablosu

```
INSERT INTO bitki_sagligi.VakaKaydi (KullaniciID, BitkiID, TedaviID) VALUES
(1,1,1), (2,2,2), (3,3,3), (4,4,4), (5,5,5),
(6,6,6), (7,7,7), (8,8,8), (9,9,9), (10,10,10),
(11,11,11), (12,12,12), (13,13,13), (14,14,14),
(15,15,15), (16,16,16), (17,17,17), (18,18,18),
(19,19,19), (20,20,20);
```

--Rapor Tablosu

```
INSERT INTO bitki_sagligi.Rapor (VakaID, AnalizID, KullaniciID) VALUES
(1,1,1), (2,2,2), (3,3,3), (4,4,4), (5,5,5),
(6,6,6), (7,7,7), (8,8,8), (9,9,9), (10,10,10),
(11,11,11), (12,12,12), (13,13,13), (14,14,14),
(15,15,15), (16,16,16), (17,17,17), (18,18,18),
(19,19,19), (20,20,20),
(1,21,21), (2,22,22), (3,23,23), (4,24,24), (5,25,25);
```

--Sulama Ve Gübreleme Tablosu

```
INSERT INTO bitki_sagligi.SulamaGubreleme (BitkiID, Gubre_Turu, Gubre_Dozu,
Uygulama_Suresi) VALUES
(1, 'Azotlu', 2.5, 30), (2, 'Fosforlu', 3.0, 25),
(3, 'Potasyumlu', 2.0, 20), (4, 'Organik', 4.0, 35),
(5, 'Kompoze', 3.5, 30), (6, 'Azotlu', 2.8, 28),
(7, 'Organik', 4.2, 40), (8, 'Potasyumlu', 2.3, 22),
(9, 'Fosforlu', 3.1, 26), (10, 'Azotlu', 2.6, 29),
(11, 'Organik', 4.0, 35), (12, 'Kompoze', 3.5, 30),
(13, 'Potasyumlu', 2.4, 23), (14, 'Azotlu', 2.9, 27),
(15, 'Fosforlu', 3.0, 25), (16, 'Organik', 4.1, 38),
(17, 'Kompoze', 3.6, 32), (18, 'Azotlu', 2.7, 28),
(19, 'Potasyumlu', 2.5, 24), (20, 'Organik', 4.3, 40),
(21, 'Fosforlu', 3.2, 26), (22, 'Azotlu', 2.8, 29),
(23, 'Kompoze', 3.7, 33), (24, 'Organik', 4.0, 36),
(25, 'Potasyumlu', 2.6, 25);
```

--Bir Kullanici birden fazla bitki Ekleme isterse ?

```
INSERT INTO bitki_sagligi.Bitki
(KullaniciID, LokasyonID, Bitki_Adi, Bitki_Bilimsel_Adi) VALUES
-- KullaniciID = 1 (Ahmet Yılmaz)
(1, 2, 'Arpa', 'Hordeum vulgare'),
(1, 3, 'Yulaf', 'Avena sativa'),

-- KullaniciID = 2 (Mehmet Kaya)
(2, 4, 'Soğan', 'Allium cepa'),
(2, 5, 'Sarımsak', 'Allium sativum'),

-- KullaniciID = 3 (Ayşe Demir)
(3, 6, 'Patlıcan', 'Solanum melongena'),

-- KullaniciID = 5 (Ali Can)
(5, 7, 'Ayçiçeği', 'Helianthus annuus'),
(5, 8, 'Kanola', 'Brassica napus'),

-- KullaniciID = 7 (Zeynep Arslan)
(7, 9, 'İncir', 'Ficus carica'),

-- KullaniciID = 10 (Kemal Öz)
(10, 10, 'Buğday (Kışlık)', 'Triticum aestivum'),

-- KullaniciID = 15 (Derya Polat)
(15, 11, 'Kabak', 'Cucurbita pepo'),
```

```
(15, 12, 'Salatalık', 'Cucumis sativus');
```

8. SQL KOMUTLARI VE SCRIPT DOSYALARI

8.1 Gereksinim Bazlı SQL Komutları ve Script Dosyaları

Gereksinim 1

Belirli bir ilde bulunan bitkiler, görülen hastalıkları ve patojen türleri ile birlikte listelenebilmelidir.

SQL Komutu:

```
SELECT
    b.Bitki_Adi,
    h.Hastalik_Bilimsel_Adi,
    p.Patogen_Turu
FROM bitki_sagligi.Bitki b
JOIN bitki_sagligi.Hastalik h ON b.BitkiID = h.BitkiID
JOIN bitki_sagligi.Patogen p ON h.PatogenID = p.PatogenID
JOIN bitki_sagligi.Lokasyon l ON b.LokasyonID = l.LokasyonID
WHERE l.Il = 'Adana';
```

Script Dosyası: queries/il_bazli_hastaliklar.sql

Google Drive: https://drive.google.com/file/d/1isiP_RrvDrIaqbNdHDSEAZuDBxm9-Xv8/view?usp=sharing

Gereksinim 2

Sistemde kayıtlı tüm bitkiler, bulundukları il ve ilçe bilgileriyle birlikte listelenebilmelidir.

SQL Komutu:

```
SELECT
    b.Bitki_Adi,
    b.Bitki_Bilimsel_Adi,
    l.Il,
    l.Ilce
FROM bitki_sagligi.Bitki b
JOIN bitki_sagligi.Lokasyon l

    ON b.LokasyonID = l.LokasyonID;
```

Script Dosyası : queries/bitkiler_lokasyonlari.sql

Google Drive:

https://drive.google.com/file/d/1hLQlePQyyVtYejOkYq3Uy_KFqjzT45Re/view?usp=sharing

Gereksinim 3

Hasta olan bitkiler, hastalık adları ve patojen türleriyle birlikte listelenebilmelidir.

```
SELECT
    b.Bitki_Adi,
    h.Hastalik_Bilimsel_Adi,
    p.Patogen_Turu
FROM bitki_sagligi.Bitki b
JOIN bitki_sagligi.Hastalik h
    ON b.BitkiID = h.BitkiID
JOIN bitki_sagligi.Patogen p
    ON h.PatogenID = p.PatogenID;
```

Script Dosyası: queries/hasta_bitkiler.sql

Google Drive: https://drive.google.com/file/d/1EYVhs_x_pI7YZf4bLjdVyDgaF3-9LMDI/view?usp=sharing

Gereksinim 4

Sistemde kayıtlı kullanıcılar, rollerine göre listelenebilmelidir.

```
SELECT
    AdSoyad,
    Email,
    Rol
FROM bitki_sagligi.Kullanici

ORDER BY Rol;
```

Script Dosyası: queries/kullanici_rolleri.sql

Google Drive: https://drive.google.com/file/d/186qqSAhzKfVDZuJ-Hz_KsR09iF42MKep/view?usp=sharing

Gereksinim 5

Bir bitkiye ait sulama ve gübreleme bilgileri görüntülenebilmelidir.

```
SELECT
    b.Bitki_Adi,
    sg.Gubre_Turu,
    sg.Gubre_Dozu,
    sg.Uygulama_Suresi
FROM bitki_sagligi.Bitki b
JOIN bitki_sagligi.SulamaGubreleme sg
    ON b.BitkiID = sg.BitkiID;
```

Script Dosyası: queries/sulama_gubreleme.sql

Google Drive: https://drive.google.com/file/d/1h_7u4vR7y9UF-7Kq0MFoDuJnpuSx1QVX/view?usp=sharing

Gereksinim 6

Hazırlanan raporlar, raporu oluşturan kullanıcı ile birlikte görüntülenebilmelidir.

```
SELECT
    r.RaporID,
    k.AdSoyad,
    r.AnalizID
FROM bitki_sagligi.Rapor r
JOIN bitki_sagligi.Kullanici k

    ON r.KullaniciID = k.KullaniciID;
```

Script Dosyası: queries/raporlar.sql

Google Drive:

https://drive.google.com/file/d/17WOtT6YVA_x9INdzPXL85sSeHkLWSy2-/view?usp=sharing

Gereksinim 7

Bitkilere uygulanan tedaviler ve kullanılan ilaçlar listelenebilmelidir.

```
SELECT
    b.Bitki_Adi,
    t.Tedavi_Yontemi,
    i.Ilac_Adi,
    i.Ilac_Dozaji
FROM bitki_sagligi.Bitki b
JOIN bitki_sagligi.Hastalik h
    ON b.BitkiID = h.BitkiID
JOIN bitki_sagligi.Tedavi t
    ON h.HastalikID = t.HastalikID
JOIN bitki_sagligi.Ilac i
    ON t.TedaviID = i.TedaviID;
```

Script Dosyası: queries/ tedavi_ve_ilacilar.sql

Google Drive:

https://drive.google.com/file/d/10ILs1qK0ExAzg4Y7mZhxf_4BIqtOv7F9/view?usp=sharing

9. SAKLI YORDAM VE TETİKLEYİCİ

9.1 Saklı Yordam Adı (Stored Procedure)

9.1.1.1 Amaç ve İş Operasyonu

Amaç: Sistemde kayıtlı tüm bitkileri listelemek

9.1.2.1 Saklı Yordam Kodu

```
CREATE PROCEDURE bitki_sagligi.sp_TumBitkileriGetir
AS
BEGIN
    SELECT
        b.BitkiID,
        b.Bitki_Adi,
        b.Bitki_Bilimsel_Adi
    FROM bitki_sagligi.Bitki b;
END;

EXEC bitki_sagligi.sp_TumBitkileriGetir;
```

9.1.3.1 Test Senaryoları ve Sonuçları

Sonuçlar	İletiler		
BitkiID	Bitki_Adi	Bitki_Bilimsel_Adi	
1	1	Buğday	Triticum aestivum
2	2	Mısır	Zea mays
3	3	Domates	Solanum lycopersicum
4	4	Biber	Capsicum annuum
5	5	Patates	Solanum tuberosum
6	6	Pamuk	Gossypium hirsutum
7	7	Zeytin	Olea europaea
8	8	Üzüm	Vitis vinifera
9	9	Elma	Malus domestica
10	10	Arpa	Hordeum vulgare
11	11	Ayçiçeği	Helianthus annuus
12	12	Soğan	Allium cepa
13	13	Sarımsak	Allium sativum
14	14	Pirinç	Oryza sativa
15	15	Kavun	Cucumis melo
16	16	Karpuz	Citrullus lanatus
17	17	Nohut	Cicer arietinum
18	18	Mercim...	Lens culinaris
19	19	Fasulye	Phaseolus vulgaris
20	20	Çilek	Fragaria × ananassa
21	21	Pamuk	Gossypium
22	22	Limon	Citrus limon
23	23	Portakal	Citrus sinensis
24	24	Mandal...	Citrus reticulata
25	25	Nar	Punica granatum
26	26	Arpa	Hordeum vulgare
27	27	Yulaf	Avena sativa
28	28	Soğan	Allium cepa

9.1.1.2 Amaç ve İş Operasyonu

Amaç: Girilen KullaniciID'ye ait bitkileri listelemek

9.1.2.2 Saklı Yordam Kodu

```
CREATE PROCEDURE bitki_sagligi.sp_KullaniciBitkileriGetir
    @KullaniciID INT
AS
BEGIN
    SELECT
        b.BitkiID,
```

```

        b.Bitki_Adi,
        b.Bitki_Bilimsel_Adi
    FROM bitki_sagligi.Bitki b
    WHERE b.KullaniciID = @KullaniciID;
END;

EXEC bitki_sagligi.sp_KullaniciBitkileriGetir @KullaniciID = 15;

```

9.1.3.2 Test Senaryoları ve Sonuçları

Sonuçlar		İletiler	
	BitkiID	Bitki_Adi	Bitki_Bilimsel_Adi
1	15	Kavun	Cucumis melo
2	35	Kabak	Cucurbita pepo
3	36	Salatalık	Cucumis sativus

9.1.1.3 Amaç ve İş Operasyonu

Amaç: Henüz hastalık kaydı bulunmayan (sağlıklı) bitkileri listelemek

9.1.2.3 Saklı Yordam Kodu

```

CREATE PROCEDURE bitki_sagligi.sp_SaglikliBitkileriGetir
AS
BEGIN
    SELECT
        b.BitkiID,
        b.Bitki_Adi
    FROM bitki_sagligi.Bitki b
    LEFT JOIN bitki_sagligi.Hastalik h
        ON h.BitkiID = b.BitkiID
    WHERE h.HastalikID IS NULL;
END;

EXEC bitki_sagligi.sp_SaglikliBitkileriGetir;

```

9.1.3.3 Test Senaryoları ve Sonuçları

Sonuçlar		İletiler
	BitkiID	Bitki_Adi
1	21	Pamuk
2	22	Limon
3	23	Portakal
4	24	Mandalina
5	25	Nar
6	26	Arpa
7	27	Yulaf
8	28	Soğan
9	29	Sarımsak
10	30	Patlıcan
11	31	Ayçiçeği
12	32	Kanola
13	33	İncir
14	34	Buğday (...)
15	35	Kabak
16	36	Salatalık

9.1.1.4 Amaç ve İş Operasyonu

Amaç: Yeni bir rapor kaydı eklemek

Parametreler:

@VakaID → İlgili vaka kaydı

@AnalizID → Kullanılan toprak analizi

@KullaniciID → Raporu oluşturan kullanıcı

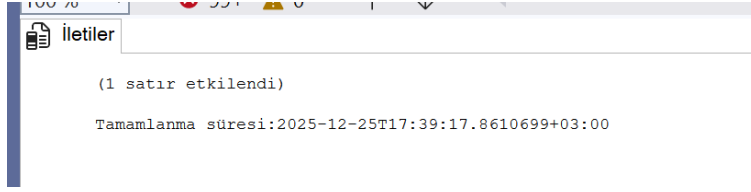
Kullanım Alanı: Rapor oluşturma işlemleri

9.1.2.4 Saklı Yordam Kodu

```
CREATE PROCEDURE bitki_sagligi.sp_RaporEkle
    @VakaID INT,
    @AnalizID INT,
    @KullaniciID INT
AS
BEGIN
    INSERT INTO bitki_sagligi.Rapor (VakaID, AnalizID, KullaniciID)
    VALUES (@VakaID, @AnalizID, @KullaniciID);
END;

EXEC bitki_sagligi.sp_RaporEkle
    @VakaID = 1,
    @AnalizID = 11,
    @KullaniciID = 4;
```

9.1.3.4 Test Senaryoları ve Sonuçları



9.1.1.5 Amaç ve İş Operasyonu

Amaç: Bitkinin hasar görülen bölgesine ait analiz sonuçlarınınıdetaylı şekilde listelemek

Kullanım Alanı:

Teknik analiz

Akademik raporlar

9.1.2.5 Saklı Yordam Kodu

```
CREATE PROCEDURE bitki_sagligi.sp_BolgeBazliAnalizler
AS
BEGIN
    SELECT
        b.Bitki_Adi,
        bb.Bitki_Bolge_Adi,
        ta.pH,
        ta.Azot,
        ta.Fosfor,
        ta.Potasyum
    FROM bitki_sagligi.ToprakAnalizi ta
    JOIN bitki_sagligi.BitkiBolgesi bb
        ON bb.BolgeID = ta.BolgeID
    JOIN bitki_sagligi.Bitki b
        ON b.BitkiID = bb.BitkiID;
END;

EXEC bitki_sagligi.sp_BolgeBazliAnalizler;
```

9.1.3.5 Test Senaryoları ve Sonuçları

	Bitki_Adi	Bitki_Bolge_Adi	pH	Azot	Fosfor	Potasyum
1	Buğday	Yaprak	6,5	2,1	1,3	2
2	Mısır	Gövde	6,8	2,3	1,5	2,2
3	Domates	Kök	7	1,9	1,2	1,8
4	Biber	Yaprak	6,6	2	1,4	2,1
5	Patates	Gövde	6,7	2,2	1,6	2,3
6	Pamuk	Kök	6,4	1,8	1,1	1,7
7	Zeytin	Yaprak	6,9	2,4	1,7	2,4
8	Üzüm	Gövde	7,1	2	1,3	2
9	Elma	Kök	6,3	1,7	1	1,6
10	Arpa	Yaprak	6,8	2,1	1,4	2,2
11	Ayçiçeği	Gövde	7	2,5	1,8	2,6
12	Soğan	Kök	6,6	2	1,3	2,1
13	Sarımsak	Yaprak	6,9	2,3	1,6	2,4
14	Pirinç	Gövde	7,2	2,6	1,9	2,7
15	Kavun	Kök	6,5	1,9	1,2	1,9
16	Karpuz	Yaprak	6,7	2,2	1,5	2,3
17	Nohut	Gövde	6,8	2,1	1,4	2,2
18	Mercim...	Kök	6,4	1,8	1,1	1,7
19	Fasulye	Yaprak	7	2,4	1,7	2,5
20	Çilek	Gövde	6,6	2	1,3	2,1
21	Pamuk	Kök	6,9	2,3	1,6	2,4
22	Limon	Yaprak	7,1	2,5	1,8	2,6
23	Portakal	Gövde	6,7	2,1	1,4	2,2
24	Mandal...	Kök	6,8	2,2	1,5	2,3
25	Nar	Yaprak	7	2,4	1,7	2,5

9.1.1.6 Amaç ve İş Operasyonu

Amaç: Rapor + Vaka + Hastalık + Tedavi + ToprakAnalizi bilgilerini birleştirip basit kurallarla "RiskSkoru" ve "RiskSeviyesi" üretmek.

Basit risk kuralları (örnek):

pH < 5.5 veya pH > 7.5 => +2 puan

Azot < 1.8 => +1 puan

Fosfor < 1.2 => +1 puan

Potasyum < 1.9 => +1 puan

Tedavi süresi > 14 gün => +1 puan

RiskSeviyesi:

0-1 => Düşük

2-3 => Orta

4+ => Yüksek

9.1.2.6 Saklı Yordam Kodu

```
CREATE PROCEDURE bitki_sagligi.sp_RiskSkorluRaporlar
AS
BEGIN
    SELECT
        r.RaporID,
        k.AdSoyad AS RaporuHazirlayan,
        b.Bitki_Adi,
        h.Hastalik_Bilimsel_Adi,
        t.Tedavi_Yontemi,
        t.Tedavi_Suresi,
        ta.pH,
        ta.Azot,
        ta.Fosfor,
        ta.Potasyum,

        -- RiskSkoru hesaplama
        (
            CASE WHEN ta.pH < 5.5 OR ta.pH > 7.5 THEN 2 ELSE 0 END +
            CASE WHEN ta.Azot IS NOT NULL AND ta.Azot < 1.8 THEN 1 ELSE 0 END +
            CASE WHEN ta.Fosfor IS NOT NULL AND ta.Fosfor < 1.2 THEN 1 ELSE 0
END +
            CASE WHEN ta.Potasyum IS NOT NULL AND ta.Potasyum < 1.9 THEN 1 ELSE
0 END +
            CASE WHEN t.Tedavi_Suresi > 14 THEN 1 ELSE 0 END
        ) AS RiskSkoru,

        -- RiskSeviyesi sınıflandırma
        CASE
            WHEN (
                CASE WHEN ta.pH < 5.5 OR ta.pH > 7.5 THEN 2 ELSE 0 END +
                CASE WHEN ta.Azot IS NOT NULL AND ta.Azot < 1.8 THEN 1 ELSE 0
END +
                CASE WHEN ta.Fosfor IS NOT NULL AND ta.Fosfor < 1.2 THEN 1 ELSE
0 END +
```

```

CASE WHEN ta.Potasyum IS NOT NULL AND ta.Potasyum < 1.9 THEN 1
ELSE 0 END +
CASE WHEN t.Tedavi_Suresi > 14 THEN 1 ELSE 0 END
) >= 4 THEN 'Yüksek'
WHEN (
CASE WHEN ta.pH < 5.5 OR ta.pH > 7.5 THEN 2 ELSE 0 END +
CASE WHEN ta.Azot IS NOT NULL AND ta.Azot < 1.8 THEN 1 ELSE 0
END +
CASE WHEN ta.Fosfor IS NOT NULL AND ta.Fosfor < 1.2 THEN 1 ELSE 0
END +
CASE WHEN ta.Potasyum IS NOT NULL AND ta.Potasyum < 1.9 THEN 1
ELSE 0 END +
CASE WHEN t.Tedavi_Suresi > 14 THEN 1 ELSE 0 END
) BETWEEN 2 AND 3 THEN 'Orta'
ELSE 'Düşük'
END AS RiskSeviyesi

FROM bitki_sagligi.Rapor r
JOIN bitki_sagligi.Kullanici k ON k.KullaniciID = r.KullaniciID
JOIN bitki_sagligi.VakaKaydi vk ON vk.VakaID = r.VakaID
JOIN bitki_sagligi.Bitki b ON b.BitkiID = vk.BitkiID
JOIN bitki_sagligi.Tedavi t ON t.TedaviID = vk.TedaviID
JOIN bitki_sagligi.Hastalik h ON h.HastalikID = t.HastalikID
JOIN bitki_sagligi.ToprakAnalizi ta ON ta.AnalizID = r.AnalizID
ORDER BY RiskSkoru DESC, r.RaporID;

END;

EXEC bitki_sagligi.sp_RiskSkorluRaporlar;

```

9.1.3.6 Test Senaryoları ve Sonuçları

Sonuçlar		İletiler										
	RaporID	RaporuHazirlayan	Bitki_Adi	Hastalik_Bilimsel_Adi	Tedavi_Yontemi	Tedavi_Suresi	pH	Azot	Fosfor	Potasyum	RiskSkoru	RiskSeviyesi
1	9	Elif Yıldız	Elma	Solgunluk	Hijyen	8	6,3	1,7	1	1,6	3	Orta
2	6	Hasan Ak	Pamuk	Pas Hastalığı	İlaçlama	7	6,4	1,8	1,1	1,7	2	Orta
3	18	Onur Şimşek	Mercimek	Bakteriyel Solgunluk	Toprak İyileştirme	12	6,4	1,8	1,1	1,7	2	Orta
4	3	Ayşe Demir	Domates	Yaprak Lekesi	İlaçlama	7	7	1,9	1,2	1,8	1	Düşük
5	1	Ahmet Yılmaz	Buğday	Fusarium Solgunluğu	Fungisit	14	6,5	2,1	1,3	2	0	Düşük
6	2	Mehmet Kaya	Mısır	Kök Çürüklüğü	Toprak Drenajı	10	6,8	2,3	1,5	2,2	0	Düşük
7	4	Fatma Şahin	Biber	Gri Küf	Fungisit	10	6,6	2	1,4	2,1	0	Düşük
8	5	Ali Can	Patates	Kök Çürüklüğü	Toprak Dezenfeksiyonu	14	6,7	2,2	1,6	2,3	0	Düşük
9	7	Zeynep Arslan	Zeytin	Külleme	Kükürt	10	6,9	2,4	1,7	2,4	0	Düşük
10	8	Murat Koç	Üzüm	Antraknoz	Fungisit	12	7,1	2	1,3	2	0	Düşük
11	10	Kemal Öz	Arpa	Sürme	İlaçlama	10	6,8	2,1	1,4	2,2	0	Düşük
12	11	Burak Aydın	Ayçiçeği	Beyaz Çürüklük	Fungisit	14	7	2,5	1,8	2,6	0	Düşük
13	12	Cem Doğan	Soğan	Yumuşak Çürüklük	Bakterisit	7	6,6	2	1,3	2,1	0	Düşük
14	13	Ece Korkmaz	Sarımsak	Bakteriyel Yanıklık	Bakırli İlaç	10	6,9	2,3	1,6	2,4	0	Düşük
15	14	Serkan Uslu	Pirinç	Bronzlaşma	İlaçlama	7	7,2	2,6	1,9	2,7	0	Düşük
16	15	Derya Polat	Kavun	Mozaik	Zararlı Kontrolü	10	6,5	1,9	1,2	1,9	0	Düşük
17	16	Tolga Kurt	Karpuz	Virüs Hastalığı	İlaçlama	7	6,7	2,2	1,5	2,3	0	Düşük
18	17	Nazan Acar	Nohut	Mozaik	Virüs Kontrolü	14	6,8	2,1	1,4	2,2	0	Düşük
19	19	Hakan Yalçın	Fasulye	Yanıklık	İlaçlama	8	7	2,4	1,7	2,5	0	Düşük
20	20	Selin Er	Çilek	Sarma	Besin Takviyesi	10	6,6	2	1,3	2,1	0	Düşük
21	21	Emre Kar	Buğday	Fusarium Solgunluğu	Fungisit	14	6,9	2,3	1,6	2,4	0	Düşük
22	22	Berk Güneş	Mısır	Kök Çürüklüğü	Toprak Drenajı	10	7,1	2,5	1,8	2,6	0	Düşük
23	23	Seda Mutlu	Domates	Yaprak Lekesi	İlaçlama	7	6,7	2,1	1,4	2,2	0	Düşük
24	24	Volkan Işık	Biber	Gri Küf	Fungisit	10	6,8	2,2	1,5	2,3	0	Düşük
25	25	Merve Toprak	Patates	Kök Çürüklüğü	Toprak Dezenfeksiyonu	14	7	2,4	1,7	2,5	0	Düşük
26	26	Fatma Şahin	Buğday	Fusarium Solgunluğu	Fungisit	14	7	2,5	1,8	2,6	0	Düşük

9.2 Tetikleyici (Trigger)

9.1.1 Amaç ve Tetikleme Koşulu

Amaç : ToprakAnalizi pH kontrol tetikleyicisi, toprak analizlerinde girilen pH değerlerinin geçerli aralıkta(0-14) olmasını sağlayarak veri bütünlüğünün düzenli olmasını sağlıyor.

9.1.2 Tetikleyici Kodu

[Tetikleyici kodunu ve açıklamasını yazınız.]

ToprakAnalizi tablosu üzerinde gerçekleştirilen **INSERT** ve **UPDATE** işlemlerinden sonra otomatik olarak çalışmaktadır. Tetikleyici, bu tabloyu kullanarak yeni girilen pH değerlerini kontrol eder. Eğer pH değeri **0 ile 14 aralığı dışında** ise, tetikleyici hata mesajı üretir ve ilgili işlemi geri alır (**ROLLBACK**). Böylece hatalı pH değeri veritabanına kaydedilmez.

```
CREATE OR ALTER TRIGGER bitki_sagligi.tr_ToprakAnalizi_pH_Kontrol
ON bitki_sagligi.ToprakAnalizi
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM inserted WHERE pH < 0 OR pH > 14)
    BEGIN
        RAISERROR('pH 0 ile 14 arasında olmalıdır.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END
END;
```

```
UPDATE bitki_sagligi.ToprakAnalizi SET pH = 20 WHERE AnalizID = 1; -- hatalı
UPDATE bitki_sagligi.ToprakAnalizi SET pH = 6.8 WHERE AnalizID = 1; -- başarılı
```

9.1.3 Test Senaryoları ve Sonuçları

```
UPDATE bitki_sagligi.ToprakAnalizi SET pH = 6.8 WHERE AnalizID = 1;
```

 İletiler

(1 satır etkilendi)

Tamamlanma süresi:2025-12-26T19:23:51.6014346+03:00

```
UPDATE bitki_sagligi.ToprakAnalizi SET pH = 20 WHERE AnalizID = 1;
```

 İletiler

İleti 50000, Düzey 16, Durum 1, Yordam tr_ToprakAnalizi_pH_Kontrol, Satır 10 [Toplu İş Başlangıç Satırı 16]
pH 0 ile 14 arasında olmalıdır.
İleti 3609, Düzey 16, Durum 1, Satır 17
The transaction ended in the trigger. The batch has been aborted.

Tamamlanma süresi:2025-12-26T19:37:08.0099519+03:00

9.2.1 Amaç ve Tetikleme Koşulu

Amaç : Bu tetikleyici, **Tedavi** tablosuna eklenen veya güncellenen kayıtlarda girilen **tedavi süresi** bilgisinin mantıksız şekilde negatif olmasını engellemek amacıyla oluşturulmuştur. Tedavi süresi negatif olamayacağı için, bu tür hatalı veri girişlerinin veritabanı seviyesinde önlenmesi hedeflenmiştir.

9.2.2 Tetikleyici Kodu

Bu tetikleyici, **Tedavi** tablosu üzerinde gerçekleştirilen **INSERT** ve **UPDATE** işlemlerinden sonra otomatik olarak çalışır. Tetikleyici, eklenen veya güncellenen kayıtlar üzerinden girilen **tedavi süresi** değerlerini kontrol eder.

Eğer tedavi süresi **negatif** bir değer olarak girilmişse, tetikleyici işlemi geri alır (**ROLLBACK**) ve bu kaydın veritabanına eklenmesini engeller

```
CREATE OR ALTER TRIGGER bitki_sagligi.tr_Tedavi_Sure_Kontrol
```

```

ON bitki_sagligi.Tedavi
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted
        WHERE Tedavi_Suresi < 0
    )
    BEGIN
        ROLLBACK;
        RETURN;
    END
END;

UPDATE bitki_sagligi.Tedavi SET Tedavi_Suresi = 10 WHERE TedaviID = 1;

UPDATE bitki_sagligi.Tedavi SET Tedavi_Suresi = -5 WHERE TedaviID = 1;

```

9.2.3 Test Senaryoları ve Sonuçları

```
UPDATE bitki_sagligi.Tedavi SET Tedavi_Suresi = 10 WHERE TedaviID = 1;
```

 İletiler

```

(1 satır etkilendi)

Tamamlanma süresi:2025-12-26T20:12:05.2950689+03:00

```

```
UPDATE bitki_sagligi.Tedavi SET Tedavi_Suresi = -5 WHERE TedaviID = 1;
```

 İletiler

```

İleti 3609, Düzey 16, Durum 1, Satır 21
The transaction ended in the trigger. The batch has been aborted.

Tamamlanma süresi:2025-12-26T20:13:13.9284679+03:00

```

9.3.1 Amaç ve Tetikleme Koşulu

Amaç : Bu tetikleyicinin amacı, **Hastalık** tablosunda ilişkili kaydı bulunan bitkilerin **yanlışlıkla silinmesini engellemektir**. Bir bitkiye ait hastalık bilgisi varken bu bitkinin silinmesi, veritabanındaki ilişkisel bütünlüğü bozabileceğinden, silme işleminin kontrol altına alınması hedeflenmiştir.

9.3.2 Tetikleyici Kodu

Bu tetikleyici, **Bitki** tablosu üzerinde gerçekleştirilen **DELETE** işlemlerinin yerine geçer. Silinmek istenen kayıtlar, SQL Server tarafından geçici olarak oluşturulan deleted sanal tablosunda tutulur. Tetikleyici, bu kayıtları kullanarak silinmek istenen bitkinin **Hastalık** tablosunda ilişkili bir kaydı olup olmadığını kontrol eder.

Eğer bitkiye ait bir hastalık kaydı bulunuyorsa, silme işlemi iptal edilir ve bitki veritabanında tutulmaya devam eder. Bitkinin herhangi bir hastalık kaydı bulunmaması durumunda ise silme işlemi tetikleyici tarafından gerçekleştirilir. Bu sayede hastalık bilgisi bulunan bitkilerin silinmesi engellenmiş olur.

```
CREATE OR ALTER TRIGGER bitki_sagligi.tr_Bitki_Delete_Kontrol
ON bitki_sagligi.Bitki
INSTEAD OF DELETE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM deleted d
        JOIN bitki_sagligi.Hastalik h
            ON h.BitkiID = d.BitkiID
    )
    BEGIN
        ROLLBACK;
        RETURN;
    END
    DELETE FROM bitki_sagligi.Bitki
    WHERE BitkiID IN (SELECT BitkiID FROM deleted);
END;

DELETE FROM bitki_sagligi.Bitki WHERE BitkiID = 36; --Hastalığı Olmayan Bitki

DELETE FROM bitki_sagligi.Bitki WHERE BitkiID = 5; --Hastalığı Olan Bitki
```

9.3.3 Test Senaryoları ve Sonuçları

```
DELETE FROM bitki_sagligi.Bitki WHERE BitkiID = 36; --Hastalığı Olmayan Bitki
```

 İletiler

```
(1 satır etkilendi)
```

```
(1 satır etkilendi)
```

```
Tamamlanma süresi:2025-12-26T20:24:38.2336902+03:00
```

```
DELETE FROM bitki_sagligi.Bitki WHERE BitkiID = 5; --Hastalığı Olan Bitki
```



İletiler

İleti 3609, Düzey 16, Durum 1, Satır 22

The transaction ended in the trigger. The batch has been aborted.

Tamamlanma süresi:2025-12-26T20:25:35.4017102+03:00

9.4.1 Amaç ve Tetikleme Koşulu

Bu tetikleyicinin amacı, **Kullanici** tablosuna eklenen veya güncellenen kayıtlar sırasında **geçersiz e-mail adreslerinin** veritabanına kaydedilmesini engellemektir. E-mail adresinde @ karakteri bulunmayan değerler geçerli bir e-mail formatını temsil etmediği için bu tür girişlerin kontrol altına alınması hedeflenmiştir.

9.4.2 Tetikleyici Kodu

Bu tetikleyici, **Kullanici** tablosu üzerinde gerçekleştirilen **INSERT** ve **UPDATE** işlemlerinden sonra otomatik olarak çalışır. Tetikleyici, bu tabloyu kullanarak **Email** alanını kontrol eder. Eğer e-mail adresinde @ karakteri bulunmuyorsa, tetikleyici kullanıcıya bilgilendirici bir mesaj gösterir ve işlemi geri alır (**ROLLBACK**). Böylece geçersiz e-mail adreslerinin veritabanına kaydedilmesi engellenmiş olur.

```
CREATE OR ALTER TRIGGER bitki_sagligi.tr_Kullanici_Email_Kontrol
ON bitki_sagligi.Kullanici
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted
        WHERE Email NOT LIKE '%@%'
    )
    BEGIN
        PRINT 'HATA: Gecersiz e-mail adresi. E-mail alanında @ karakteri
        bulunmalidir.';
        ROLLBACK;
        RETURN;
    END
END;

UPDATE bitki_sagligi.Kullanici SET Email = 'ahmetmail.com' WHERE KullaniciID =
1;

UPDATE bitki_sagligi.Kullanici SET Email = 'ahmet@mail.com' WHERE KullaniciID
=1;

INSERT INTO bitki_sagligi.Kullanici (AdSoyad, Email, Rol)
VALUES ('Test Kullanici', 'testmail.com', 'Ciftci');
```

9.4.3 Test Senaryoları ve Sonuçları

```
UPDATE bitki_sagligi.Kullanici SET Email = 'ahmetmail.com' WHERE KullaniciID = 1;
```

İletiler

```
HATA: Geçersiz e-mail adresi. E-mail alanında @ karakteri bulunmalıdır.
İleti 3609, Düzey 16, Durum 1, Satır 18
The transaction ended in the trigger. The batch has been aborted.

Tamamlanma süresi:2025-12-26T20:42:43.4382861+03:00
```

```
UPDATE bitki_sagligi.Kullanici SET Email = 'ahmet@mail.com' WHERE KullaniciID =1;
```

İletiler

```
(1 satır etkilendi)

Tamamlanma süresi:2025-12-26T20:43:29.0673167+03:00
```

```
INSERT INTO bitki_sagligi.Kullanici (AdSoyad, Email, Rol)
VALUES ('Test Kullanici', 'testmail.com', 'Ciftci');
```

İletiler

```
HATA: Geçersiz e-mail adresi. E-mail alanında @ karakteri bulunmalıdır.
İleti 3609, Düzey 16, Durum 1, Satır 22
The transaction ended in the trigger. The batch has been aborted.

Tamamlanma süresi:2025-12-26T20:46:42.9645046+03:00
```

9.5.1 Amaç ve Tetikleme Koşulu

Bu tetikleyicinin amacı, **Kullanici** tablosuna eklenen veya güncellenen kayıtlarda girilen **telefon numarasının 11 haneli olmasını sağlamaktır**. 11 hane dışında girilen telefon numaralarının geçersiz olacağı kabul edilerek bu tür girişlerin veritabanı seviyesinde engellenmesi hedeflenmiştir.

9.5.2 Tetikleyici Kodu

Bu tetikleyici, **Kullanici** tablosu üzerinde gerçekleştirilen **INSERT** ve **UPDATE** işlemlerinden sonra otomatik olarak çalışır. Tetikleyici, bu kayıtlar üzerinden **Telefon** alanını kontrol eder. Telefon numarası **NULL değilse** ve uzunluğu **11 haneden farklıysa**, tetikleyici kullanıcıya bilgilendirici bir mesaj gösterir ve işlemi geri alır

(ROLLBACK). Telefon alanı boş bırakılmışsa veya 11 haneliyse işlem normal şekilde tamamlanır

```
CREATE OR ALTER TRIGGER bitki_sagligi.tr_Kullanici_Telefon_11Hane_Kontrol
ON bitki_sagligi.Kullanici
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted
        WHERE Telefon IS NOT NULL
            AND LEN(Telefon) <> 11
    )
    BEGIN
        PRINT 'HATA: Telefon numarası 11 haneli olmalıdır.';
        ROLLBACK;
        RETURN;
    END
END;

UPDATE bitki_sagligi.Kullanici SET Telefon = '0532111223' WHERE KullaniciID =
1; --10 Tane

UPDATE bitki_sagligi.Kullanici SET Telefon = '05321112233' WHERE KullaniciID =
1; -- 11 hane
```

9.5.3 Test Senaryoları ve Sonuçları

```
UPDATE bitki_sagligi.Kullanici SET Telefon = '0532111223' WHERE KullaniciID =
1; --10 Tane
```

İletiler

```
HATA: Telefon numarası 11 haneli olmalıdır.
İleti 3609, Düzey 16, Durum 1, Satır 19
The transaction ended in the trigger. The batch has been aborted.

Tamamlanma süresi:2025-12-26T20:58:14.2304352+03:00
```

```
UPDATE bitki_sagligi.Kullanici SET Telefon = '05321112233' WHERE KullaniciID =
1; -- 11 hane
```

İletiler

(1 satır etkilendi)

Tamamlanma süresi:2025-12-26T20:58:48.2658020+03:00

10. TRANSACTION YÖNETİMİ

10.1 Tedavi ve İlaç Kayıtlarının Birlikte Oluşturulması

Bitki sağlığı sisteminde bir hastalık için tedavi planı oluşturma işlemi, tek başına yalnızca Tedavi tablosuna kayıt eklemekten ibaret değildir. Tedavi planının anlamlı ve uygulanabilir olabilmesi için, bu tedaviye bağlı olarak kullanılacak ilaçların ve doz bilgilerinin de sisteme kaydedilmesi gerekmektedir.

Bu nedenle tedavi oluşturma işlemi sırasında aynı anda tedavi kaydının eklenmesi ve bu tedaviye bağlı ilaç kayıtlarının oluşturulması gereklidir. Bu iki işlem birbirine bağlıdır ve **atomik olarak** gerçekleştirilmelidir. Aksi halde tedavi kaydı oluşturulup ilaç eklenemezse sistemde **ilaçsız bir tedavi**, ilaç kaydı eklenip tedavi kaydı oluşmazsa **ilişkisiz ilaç kayıtları** ortaya çıkar ve veritabanı tutarlılığı bozulur.

Bu sebeple tedavi ekleme ve ilaç ekleme işlemleri tek bir transaction içerisinde yürütülmelidir. Tüm işlemler başarıyla tamamlandığında transaction **COMMIT** edilerek veriler kalıcı hale getirilir; işlemlerden herhangi birinde hata oluşması durumunda ise yapılan tüm değişiklikler **ROLLBACK** edilerek veritabanı önceki tutarlı durumuna döndürülür.

10.2 Transaction Kodu

```
BEGIN TRY
    BEGIN TRANSACTION;

    DECLARE
        @HastalikID INT = 1,    -- Var olan bir HastalikID olmalı
        @TedaviYontemi VARCHAR(100) = 'Fungisit uygulaması',
        @TedaviSuresi INT = 14;

    -- 1) Tedavi kaydı ekle
    INSERT INTO bitki_sagligi.Tedavi (HastalikID, Tedavi_Yontemi,
    Tedavi_Suresi)
```



```

VALUES (@HastalikID, @TedaviYontemi, @TedaviSuresi);

DECLARE @YeniTedaviID INT = SCOPE_IDENTITY();

-- 2) Tedaviye bağlı ilaç kayıtlarını ekle
INSERT INTO bitki_sagligi.Ilac (TedaviID, Ilac_Adi, Ilac_Dozaji)
VALUES
    (@YeniTedaviID, 'Bakirli ilac', '2 ml/L'),
    (@YeniTedaviID, 'Koruyucu sprej', '1 ml/L');

COMMIT TRANSACTION;

SELECT
    'COMMIT edildi' AS Durum,
    @YeniTedaviID AS OlusanTedaviID;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    SELECT
        'ROLLBACK yapıldı' AS Durum,
        ERROR_NUMBER() AS HataNo,
        ERROR_MESSAGE() AS HataMesaji;
END CATCH;

```

Sonuçlar		İletiler
	Durum	OlusanTedaviID
1	COMMIT edildi	27

10.3 Başarılı Senaryo Testi

```

SELECT TOP 5 *
FROM bitki_sagligi.Tedavi
ORDER BY TedaviID DESC;

```

```

SELECT TOP 10 *
FROM bitki_sagligi.Ilac
ORDER BY IlacID DESC;

```

Sonuçlar

İletiler

	TedaviID	HastalıkID	Tedavi_Yontemi	Tedavi_Suresi
1	27	1	Fungisit uygulaması	14
2	26	1	Fungisit uygulaması	14
3	25	5	Dezenfeksiyon+	14
4	24	4	Fungisit+	10
5	23	3	İlaç+	7

	IlacID	TedaviID	Ilac_Adi	Ilac_Dozaji
1	27	27	Koruyucu sprej	1 ml/L
2	26	27	Bakirli ilaç	2 ml/L
3	25	25	SoilMax	4 ml/L
4	24	24	FungiPlus	2 ml/L
5	23	23	LeafExtra	1 ml/L
6	22	22	DrainMax	–
7	21	21	ExtraFungi	2 ml/L
8	20	20	NutriBoost	5 ml/L
9	19	19	BurnAway	2 ml/L
10	18	18	WiltStop	3 ml/L

10.4 Hata Senaryosu ve ROLLBACK Testi

SQL Komutunda Değişilen Yer:

```
DECLARE
    @HastalikID INT = 999999, -- Bilerek hatalı verildi
```

Sonuçlar		İletiler	
	Durum	HataNo	HataMesaji
1	ROLLBACK yapıldı	547	The INSERT statement conflicted with the FOREIGN ...

ROLLBACK TESTİ

```
SELECT TOP 10 *
FROM bitki_sagligi.Tedavi
ORDER BY TedaviID DESC;
```

İletiler

İleti 208, Düzey 16, Durum 1, Satır 78
Invalid object name 'bitki_sagligi.Teda'.

Tamamlanma süresi:2025-12-26T21:31:01.4944152+03:00

11.1 Lokasyon Oluşturma – Bitki Kaydı Oluşturma

Sisteme yeni lokasyon girilirken aynı anda o lokasyonda kullanıcıya ait bitki kaydı da oluşturulur. Yani lokasyon kaydı ekleme ve bitki kaydı ekleme birlikte yürütülür. Bitki eklenemezse lokasyon tek başına kalmasın diye transaction kullanılır.

11.2 Transaction Kodu

```
BEGIN TRY
    BEGIN TRANSACTION

    INSERT INTO bitki_sagligi.Lokasyon (Il, Ilce)
    VALUES ('TestIl', CONCAT('TestIlce_', LEFT(CONVERT(VARCHAR(36), NEWID()),
8)));

    DECLARE @YeniLokasyonID INT = SCOPE_IDENTITY();

    INSERT INTO bitki_sagligi.Bitki (KullaniciID, LokasyonID, Bitki_Adi,
Bitki Bilimsel_Adi)
    VALUES (1, @YeniLokasyonID, 'Test Bitki', 'Testus bitkius');

    COMMIT;

    SELECT 'COMMIT' AS Durum, @YeniLokasyonID AS LokasyonID;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    SELECT 'ROLLBACK' AS Durum, ERROR_MESSAGE() AS HataMesaji;
END CATCH;
```

Sonuçlar		İletiler
	Durum	LokasyonID
1	COMMIT	30

11.3 Başarılı Senaryo Testi

```
SELECT TOP 5 *
FROM bitki_sagligi.Lokasyon
ORDER BY LokasyonID DESC;
```

```
SELECT TOP 5 *
FROM bitki_sagligi.Bitki
```

```
ORDER BY BitkiID DESC;
```

 Sonuçlar  İletiler

	LokasyonID	İl	İlce
1	30	Testİl	Testİlce_D0485A13
2	26	Testİl	Testİlce_585B7A8A
3	25	Hatay	Kırıkhan
4	24	Mardin	Kızıltepe
5	23	Diyarbakır	Bismil

	BitkiID	KullaniciID	LokasyonID	Bitki_Adi	Bitki_Bilimsel_Adi
1	41	1	30	Test Bitki	Testus bitkius
2	37	1	26	Test Bitki	Testus bitkius
3	35	15	11	Kabak	Cucurbita pepo
4	34	10	10	Buğday...	Triticum aestivum
5	33	7	9	İncir	Ficus carica

11.4 Hata Senaryosu ve ROLLBACK Testi

```
INSERT INTO bitki_sagligi.Bitki (KullaniciID, LokasyonID, Bitki_Adi,
Bitki_Bilimsel_Adi)
VALUES (999999, @YeniLokasyonID, 'Test Bitki', 'Testus bitkius'); --Bu
şekilde KullaniciID değiştirelim
```

 Sonuçlar  İletiler

	Durum	HataMesaji
1	ROLLBACK	The INSERT statement conflicted with the FOREIGN ...

ROLLBACK TESTİ

```
SELECT TOP 5 *
FROM bitki_sagligi.Lokasyon
WHERE İlce LIKE 'HATA %'
ORDER BY LokasyonID DESC;
```

 Sonuçlar  İletiler

	LokasyonID	İl	İlce
--	------------	----	------

12.1 Patogen Oluşturma – Hastalık Oluşturma

Sisteme yeni bir patojen eklenirken, aynı anda bu patojene bağlı bir hastalık kaydı da oluşturulur. Yani patojen kaydı ekleme ve hastalık kaydı oluşturma işlemleri birlikte yürütülür. Hastalık eklenemezse patojenin tek başına kalması istenmez; bu yüzden işlem transaction içinde yapılır. Başarılıysa **COMMIT**, hata olursa **ROLLBACK** uygulanır.

12.2 Transaction Kodu

```
SET NOCOUNT ON;

DECLARE @Durum VARCHAR(10);
DECLARE @PatogenID INT;

BEGIN TRY
    BEGIN TRANSACTION;

    INSERT INTO bitki_sagligi.Patogen (Patogen_Adi, Patogen_Turu)
    VALUES ('COMMIT_PATOGEN_002', 'Mantar');

    SET @PatogenID = SCOPE_IDENTITY();

    COMMIT TRANSACTION;
    SET @Durum = 'COMMIT';

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SET @Durum = 'ROLLBACK';
END CATCH;

SELECT @Durum AS Durum, @PatogenID AS PatogenID;
```

Sonuçlar		İletiler
	Durum	PatogenID
1	COMMIT	34

12.3 Başarılı Senaryo Testi

```
SELECT *
FROM bitki_sagligi.Patogen
WHERE Patogen_Adi = 'COMMIT_PATOGEN_002';
```

Sonuçlar		İletiler	
	PatogenID	Patogen_Adi	Patogen_Turu
1	34	COMMIT_PATOGEN_002	Mantar

12.4 Hata Senaryosu ve ROLLBACK Testi

```
-- KASITLI HATA: rollback'i tetiklemek için
THROW 51010, 'Test için bilincli hata (ROLLBACK senaryosu).', 1;
```

Sonuçlar		İletiler	
	Durum	PatogenID	
1	ROLLBACK	35	

ROLLBACK HATASI

```
SELECT *
FROM bitki_sagligi.Patogen
WHERE Patogen_Adi = 'HATA_PATOGEN_003';
```

Sonuçlar		İletiler	
	PatogenID	Patogen_Adi	Patogen_Turu

11. TAKIM ÇALIŞMASI VE GÖREV DAĞILIMI

11.1 Takım Üyeleri ve Roller

Takım Üyesi	Öğrenci No	Temel Rol
Zeynep Sevim Özkök	235260006	[Proje Lideri / Veri Tabanı Tasarımcısı]
Ahmet KURT	245260018	[SQL Geliştirici / Test Sorumlusu]
Ecenur ÇEVİK	245260020	[Dokümantasyon / Kalite Kontrol]

11.2 Gerçekleştirilen İşler ve Sorumluluk Matrisi

[Aşağıdaki tabloda projede gerçekleştirilen her iş için talimat veren, gerçekleştiren ve kontrol eden kişiler belirtilecektir.]

No	Yapılan İş / Özellik	Talimat Veren	Gerçekleştiren	Kontrol Eden	Durum
1	Proje gereksinimlerinin belirlenmesi	Zeynep Sevim ÖZKÖK	Ecenur ÇEVİK	Ahmet KURT	✓
2	E-R diyagramının oluşturulması	Ahmet KURT	Zeynep Sevim ÖZKÖK	Ecenur ÇEVİK	✓
3	İlişkisel şemalara dönüştürme	Ecenur ÇEVİK	Zeynep Sevim ÖZKÖK	Ahmet KURT	✓
4	Normalizasyon (3NF/BCNF)	Zeynep Sevim ÖZKÖK	Ahmet KURT	Ecenur ÇEVİK	✓
5	SQL Server'da şema oluşturma	Ecenur ÇEVİK	Ahmet KURT	Zeynep Sevim ÖZKÖK	✓
6	Birincil/Yabancı anahtar tanımları	Ahmet KURT	Ecenur ÇEVİK	Zeynep Sevim ÖZKÖK	✓
7	Nitelik kısıtlamalarının eklenmesi	Zeynep Sevim ÖZKÖK	Ecenur ÇEVİK	Ecenur ÇEVİK	✓
8	Örnek verilerin eklenmesi	Ecenur ÇEVİK	Ahmet KURT	Zeynep Sevim ÖZKÖK	✓
9	SQL sorgu script dosyalarının hazırlanması	Zeynep Sevim ÖZKÖK	Ahmet KURT	Ecenur ÇEVİK	✓

10	Saklı yordamın (Stored Procedure) yazılması	Zeynep Sevim ÖZKÖK	Ahmet KURT	Ecenur ÇEVİK	✓
11	Tetikleyicinin (Trigger) yazılması	Zeynep Sevim ÖZKÖK	Ahmet KURT	Ahmet KURT	✓
12	Transaction yönetimi kodunun yazılması	Ahmet KURT	Zeynep Sevim ÖZKÖK	Ecenur ÇEVİK	✓
13	COMMIT/ROLLBACK test senaryoları	Zeynep Sevim ÖZKÖK	Ecenur ÇEVİK	Ahmet KURT	✓
14	Rapor hazırlama ve dokümantasyon	Ahmet KURT	Zeynep Sevim ÖZKÖK	Ecenur ÇEVİK	✓
15	Son kontrol ve teslim	Ecenur ÇEVİK	Ahmet KURT	Zeynep Sevim ÖZKÖK	✓

Not: Talimat Veren: İlgili işin yapılması gerektiğine karar veren ve yönlendiren kişi.
Gerçekleştiren: İşin fiilen yapan kişi. Kontrol Eden: İşin doğruluğunu ve kalitesini kontrol eden kişi.

12. SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında geliştirilen **Bitki Hastalıkları Takip Sistemi**, veri tabanı sistemlerinin analiz, tasarım ve uygulama aşamalarını bütüncül bir şekilde ele alan kapsamlı bir çalışma olmuştur. Proje sürecinde, tarımsal üretimde karşılaşılan bitki hastalıklarının kayıt altına alınması, hastalıklara ait patojen ve belirti bilgilerinin tutulması, uygulanan tedavi ve kullanılan ilaçların izlenmesi ile birlikte toprak analiz sonuçlarının değerlendirilmesini sağlayan bir veri tabanı altyapısı başarıyla oluşturulmuştur.

Projenin ilk aşamasında sistem gereksinimleri belirlenmiş, fonksiyonel ve fonksiyonel olmayan gereksinimler net bir şekilde tanımlanmıştır. Bu gereksinimler doğrultusunda oluşturulan **E-R diyagramı**, sistemde yer alan varlıklar arasındaki ilişkileri açık bir biçimde ortaya koymuş ve ilişkisel şemalara dönüştürülmüştür. Ardından gerçekleştirilen **normalizasyon çalışmaları**, veri tekrarını ve tutarsızlığı önleyerek veri bütünlüğünü sağlayacak bir yapı oluşturmuştur. Özellikle BCNF düzeyine kadar yapılan normalizasyon işlemleri, veri tabanının uzun vadede sağlıklı ve sürdürülebilir olmasına katkı sağlamıştır.

SQL Server üzerinde oluşturulan tablolar, birincil ve yabancı anahtarlar aracılığıyla ilişkilendirilmiş; referans bütünlüğü kuralları etkin şekilde uygulanmıştır. Örnek veri girişleri ve gereksinim bazlı SQL sorguları sayesinde sistemin işlevselliği test edilmiştir. Ayrıca geliştirilen **saklı yordamlar**, sistemde sık kullanılan işlemlerin daha düzenli ve performanslı bir şekilde gerçekleştirilmesini sağlamıştır. **Tetikleyiciler (trigger)** aracılığıyla hatalı veri girişleri engellenmiş, veri bütünlüğü ve iş kuralları veritabanı seviyesinde güvence altına alınmıştır.

Transaction yönetimi kapsamında, özellikle tedavi ve ilaç kayıtlarının birlikte oluşturulması gibi kritik işlemler **COMMIT ve ROLLBACK** mekanizmaları kullanılarak kontrol altına alınmıştır. Bu sayede hata oluşması durumunda sistemin tutarlı bir duruma geri dönebilmesi sağlanmıştır. Gerçekleştirilen test senaryoları, oluşturulan veri tabanı yapısının doğru ve güvenilir şekilde çalıştığını göstermiştir.

Sonuç olarak, bu proje ile veri tabanı tasarımı, normalizasyon, SQL sorguları, saklı yordamlar, tetikleyiciler ve transaction yönetimi gibi temel veri tabanı kavramları uygulamalı olarak öğrenilmiş ve pekiştirilmiştir. Geliştirilen sistem, akademik düzeyde beklenen tüm gereksinimleri karşılamakta olup, ileride gerçek zamanlı sensör verileri veya otomatik hastalık teşhisi gibi ek modüllerle genişletilmeye uygun bir altyapı sunmaktadır.